

.NET 10 App Dev Hands-On Lab

Razor Pages/MVC/API Lab 2a – Common Services

This lab builds the shared services used by the ASP.NET Core applications. Before starting this lab, you must have completed the first lab for one of the applications: Razor Pages, MVC, or API. **This entire lab works in the `AutoLot.Services` project.**

Copilot Agent Mode

- Copy the file `Lab_Web02a.md` from the student repo, and execute the prompts. When complete, compare the generated code with Parts 1, 2, and 3. Then resume with the section label Copilot Agent Mode and Manul

Manual

Start by renaming the `Class1.cs` file to `GlobalUsings.cs`. Update the code to the following:

```
global using AutoLot.Dal.Repos;
global using AutoLot.Dal.Repos.Base;
global using AutoLot.Dal.Repos.Interfaces;
global using AutoLot.Dal.Repos.Interfaces.Base;

global using Microsoft.AspNetCore.Builder;
global using Microsoft.Extensions.DependencyInjection;
global using Microsoft.Extensions.Configuration;
global using Microsoft.Extensions.Hosting;
global using Microsoft.Extensions.Logging;

global using Serilog;
global using Serilog.Context;
global using Serilog.Core.Enrichers;
global using Serilog.Events;
global using Serilog.Sinks.MSSqlServer;

global using System.ComponentModel;
global using System.ComponentModel.DataAnnotations;

global using System.Data;
global using System.Diagnostics;

global using System.Runtime.CompilerServices;
```

Part 1: Add Logging Support

Step 1: Add the Logging Settings View Model

- Add a new folder named Logging in the AutoLot.Services project. In that folder, add a new folder named Settings, and in that folder, add a new class file named AppLoggingSettings.cs. Update the class code to the following:

```
namespace AutoLot.Services.Logging.Settings;
```

```
public class AppLoggerSettings
{
    [Required]
    public GeneralSettings General { get; set; }
    [Required]
    public FileSettings File { get; set; }
    [Required]
    public SqlServerSettings MsSqlServer { get; set; }
    public class GeneralSettings
    {
        [Required(AllowEmptyStrings = false)]
        public string RestrictedToMinimumLevel { get; set; }
    }
    public class SqlServerSettings
    {
        [Required(AllowEmptyStrings = false)]
        public string TableName { get; set; }
        [Required(AllowEmptyStrings = false)]
        public string Schema { get; set; }
        [Required(AllowEmptyStrings = false)]
        public string ConnectionStringName { get; set; }
    }
    public class FileSettings
    {
        [Required(AllowEmptyStrings = false)]
        public string Drive { get; set; }
        [Required(AllowEmptyStrings = false)]
        public string FilePath { get; set; }
        [Required(AllowEmptyStrings = false)]
        public string FileName { get; set; }

        public string FullLogPathAndFileName =>
        $"{Drive}{Path.VolumeSeparatorChar}{Path.DirectorySeparatorChar}{FilePath}{Path.DirectorySeparatorChar}{FileName}";
    }
}
```

Step 2: Add the Logging Interface

- In the Logging folder, add a new folder named Interfaces; in that folder, add a new interface file named IAppLogging.cs. Update the interface code to the following:

```
namespace AutoLot.Services.Logging.Interfaces;

public interface IAppLogger
{
    void LogAppError(Exception exception, string message,
        [CallerMemberName] string memberName = "",
        [CallerFilePath] string filePath = "",
        [CallerLineNumber] int lineNumber = 0);
    void LogAppError(string message,
        [CallerMemberName] string memberName = "",
        [CallerFilePath] string filePath = "",
        [CallerLineNumber] int lineNumber = 0);
    void LogAppCritical(Exception exception, string message,
        [CallerMemberName] string memberName = "",
        [CallerFilePath] string filePath = "",
        [CallerLineNumber] int lineNumber = 0);
    void LogAppCritical(string message,
        [CallerMemberName] string memberName = "",
        [CallerFilePath] string filePath = "",
        [CallerLineNumber] int lineNumber = 0);
    void LogAppDebug(string message,
        [CallerMemberName] string memberName = "",
        [CallerFilePath] string filePath = "",
        [CallerLineNumber] int lineNumber = 0);
    void LogAppTrace(string message,
        [CallerMemberName] string memberName = "",
        [CallerFilePath] string filePath = "",
        [CallerLineNumber] int lineNumber = 0);
    void LogAppInformation(string message,
        [CallerMemberName] string memberName = "",
        [CallerFilePath] string filePath = "",
        [CallerLineNumber] int lineNumber = 0);
    void LogAppWarning(string message,
        [CallerMemberName] string memberName = "",
        [CallerFilePath] string filePath = "",
        [CallerLineNumber] int lineNumber = 0);
}
```

- Add the following to the GlobalUsings.cs file:

```
global using AutoLot.Services.Logging;
global using AutoLot.Services.Logging.Interfaces;
global using AutoLot.Services.Logging.Settings;
```

Step 3: Add the Logging Implementation

- In the Logging folder, add a class file named AppLogging.cs. Update the code to the following:

```
namespace AutoLot.Services.Logger;
public sealed class AppLogger(ILogger<AppLogger> logger) : IAppLogger
{
    internal void LogWithException(
        string memberName, string filePath, int lineNumber, string message, Exception ex,
        Action<Exception, string, object[]> logAction)
    {
        var disposables =
            new List<IDisposable>
            {
                LogContext.PushProperty("MemberName", memberName),
                LogContext.PushProperty("FilePath", filePath),
                LogContext.PushProperty("LineNumber", lineNumber)
            };
        try
        {
            logAction(ex, message, Array.Empty<object>());
        }
        finally
        {
            foreach (var d in disposables)
            {
                d.Dispose();
            }
        }
    }
    internal void LogWithoutException(
        string memberName, string filePath, int lineNumber, string message,
        Action<string, object[]> logAction)
    {
        var disposables =
            new List<IDisposable>
            {
                LogContext.PushProperty("MemberName", memberName),
                LogContext.PushProperty("FilePath", filePath),
                LogContext.PushProperty("LineNumber", lineNumber)
            };
        try
        {
            logAction(message, Array.Empty<object>());
        }
        finally
        {
            foreach (var d in disposables)
            {
                d.Dispose();
            }
        }
    }
}
```

```

public void LogAppError(
    Exception exception, string message, [CallerMemberName] string memberName = "",
    [CallerFilePath] string filePath = "", [CallerLineNumber] int lineNumber = 0)
    => LogWithException(memberName, filePath, lineNumber, message, exception, logger.LogError);

public void LogAppError(
    string message, [CallerMemberName] string memberName = "",
    [CallerFilePath] string filePath = "", [CallerLineNumber] int lineNumber = 0)
    => LogWithoutException(memberName, filePath, lineNumber, message, logger.LogError);

public void LogAppCritical(
    Exception exception, string message, [CallerMemberName] string memberName = "",
    [CallerFilePath] string filePath = "", [CallerLineNumber] int lineNumber = 0)
    => LogWithException(memberName, filePath, lineNumber, message, exception,
        logger.LogCritical);

public void LogAppCritical(
    string message, [CallerMemberName] string memberName = "",
    [CallerFilePath] string filePath = "", [CallerLineNumber] int lineNumber = 0)
    => LogWithoutException(memberName, filePath, lineNumber, message, logger.LogCritical);

public void LogAppDebug(
    string message, [CallerMemberName] string memberName = "",
    [CallerFilePath] string filePath = "", [CallerLineNumber] int lineNumber = 0)
    => LogWithoutException(memberName, filePath, lineNumber, message, logger.LogDebug);

public void LogAppTrace(
    string message, [CallerMemberName] string memberName = "",
    [CallerFilePath] string filePath = "", [CallerLineNumber] int lineNumber = 0)
    => LogWithoutException(memberName, filePath, lineNumber, message, logger.LogTrace);

public void LogAppInformation(
    string message, [CallerMemberName] string memberName = "",
    [CallerFilePath] string filePath = "", [CallerLineNumber] int lineNumber = 0)
    => LogWithoutException(memberName, filePath, lineNumber, message, logger.LogInformation);

public void LogAppWarning(
    string message, [CallerMemberName] string memberName = "",
    [CallerFilePath] string filePath = "", [CallerLineNumber] int lineNumber = 0)
    => LogWithoutException(memberName, filePath, lineNumber, message, logger.LogWarning);
}

```

Part 2: Add the String Utility Extension Block

- Add a new folder named Utilities in the AutoLot.Services project and, in that folder, add a new class file named StringExtensions.cs. Update the code to match the following:

```
namespace AutoLot.Services.Utilities;
public static class StringExtensions2
{
    extension (string value)
    {
        public string RemoveControllerSuffix()
            => value != null && value.EndsWith("Controller", StringComparison.OrdinalIgnoreCase)
                ? value[..^10] : value;
        public string RemoveAsyncSuffix()
            => value != null && value.EndsWith("Async", StringComparison.Ordinal)
                ? value[..^5] : value;
    }
}
```

- Add the following to the GlobalUsings.cs file:

```
global using AutoLot.Services.Utilities;
```

Part 3: Add the SimpleService Interface and Classes

- Create a new folder named Simple, and in that folder, add a new folder named Interfaces. In that folder, create a new interface named ISimpleService.cs and update the contents to the following:

```
namespace AutoLot.Services.Simple.Interfaces;
public interface ISimpleService
{
    string SayHello();
}
```

- Add the following to the GlobalUsings.cs file:

```
global using AutoLot.Services.Simple;
global using AutoLot.Services.Simple.Interfaces;
```

- Create two new classes named SimpleServiceOne and SimpleServiceTwo, and update them to the following:

```
//SimpleServiceOne
namespace AutoLot.Services.Simple;
public class SimpleServiceOne : ISimpleService
{
    public string SayHello() => "Hello from SimpleServiceOne!";
}
//SimpleServiceTwo
namespace AutoLot.Services.Simple;
public class SimpleServiceTwo : ISimpleService
{
    public string SayHello() => "Hello from SimpleServiceTwo!";
}
```

Part 4: Add the DealerInfo ViewModel

- In the AutoLot.Services project, create a new folder named ViewModels, and in that folder, create a new file named DealerInfo.cs and update the contents to the following:

```
namespace AutoLot.Services.ViewModels;
public class DealerInfo
{
    public string DealerName { get; set; }
    public string City { get; set; }
    public string State { get; set; }
}
```

- Add the following to the GlobalUsings.cs file:

```
global using AutoLot.Services.ViewModels;
```

Copilot Agent Mode and Manual

Step 1: Add the Logging Configuration Extension Method

- Create a new folder named Configuration in the Logging folder and add a new class named LoggingConfiguration.cs. Update the code to the following:

```
using Serilog.Debugging;
namespace AutoLot.Services.Logging.Configuration;

public static class LoggingConfiguration
{
    public static IServiceCollection RegisterLoggingInterfaces(this IServiceCollection services)
    {
        services.AddScoped<IAppLogger, AppLogger>();
        return services;
    }

    private static readonly string OutputTemplate =
        @"[{Timestamp:yy-MM-dd HH:mm:ss}
{Level}] {ApplicationName}: {SourceContext} {NewLine} Message: {Message} {NewLine} in method {MemberName}
at {FilePath}: {LineNumber} {NewLine} {Exception} {NewLine}";

    private static readonly ColumnOptions ColumnOptions = new()
    {
        AdditionalColumns = new List<SqlColumn>
        {
            new() { DataType = SqlDbType.VarChar, ColumnName = "ApplicationName" },
            new() { DataType = SqlDbType.VarChar, ColumnName = "MachineName" },
            new() { DataType = SqlDbType.VarChar, ColumnName = "MemberName" },
            new() { DataType = SqlDbType.VarChar, ColumnName = "FilePath" },
            new() { DataType = SqlDbType.Int, ColumnName = "LineNumber" },
            new() { DataType = SqlDbType.VarChar, ColumnName = "SourceContext" },
            new() { DataType = SqlDbType.VarChar, ColumnName = "RequestPath" },
            new() { DataType = SqlDbType.VarChar, ColumnName = "ActionName" }
        }
    };
};
```

```

public static void ConfigureSerilog(
    this WebApplicationBuilder builder, IConfiguration configuration)
{
    ConfigureSerilogInternal(builder, configuration);
}
internal static void ConfigureSerilogInternal(
    IHostApplicationBuilder builder,
    IConfiguration configuration)
{
    builder.Logging.ClearProviders();
    AppLoggerSettings settings =
        configuration.GetSection(nameof(AppLoggerSettings)).Get<AppLoggerSettings>();
    ValidateSettings(settings);
    string connectionStringName = settings.MsSqlServer.ConnectionStringName;
    string connectionString = configuration.GetConnectionString(connectionStringName);
    if (string.IsNullOrEmpty(connectionString))
    {
        throw new CustomException(
            $"Connection string '{connectionStringName}' referenced by " +
            $"{nameof(AppLoggerSettings)}.{nameof(AppLoggerSettings.MsSqlServer)}.{nameof(AppLoggerSettings.S
qlServerSettings.ConnectionStringName)}' is missing or empty.");
    }
    string tableName = settings.MsSqlServer.TableName;
    string schema = settings.MsSqlServer.Schema;
    string restrictedToMinimumLevel = settings.General.RestrictedToMinimumLevel;
    if (!Enum.TryParse<LogEventLevel>(restrictedToMinimumLevel, out LogEventLevel logLevel))
    {
        logLevel = LogEventLevel.Debug;
    }
    MSSqlServerSinkOptions sqlOptions = new()
    {
        AutoCreateSqlTable = false, SchemaName = schema, TableName = tableName
    };
    if (builder.Environment.IsDevelopment())
    {
        sqlOptions.BatchPeriod = new TimeSpan(0, 0, 0, 1);
        sqlOptions.BatchPostingLimit = 1;
    }
    LoggerConfiguration log = new LoggerConfiguration().
        MinimumLevel.Is(logLevel).MinimumLevel
        .Override("Microsoft", LogEventLevel.Error)
        .Enrich.FromLogContext()
        .Enrich.With(new PropertyEnricher(
            "ApplicationName", configuration.GetValue<string>("ApplicationName")))
        .Enrich.WithMachineName()
        .WriteTo .File(builder.Environment.IsDevelopment() ? settings.File.FileName
            : settings.File.FullLogPathAndFileName, // "ErrorLog.txt",
            rollingInterval: RollingInterval.Day,
            restrictedToMinimumLevel: logLevel,
            outputTemplate: OutputTemplate)
        .WriteTo.Console(logLevel)
        .WriteTo.MSSqlServer(connectionString, sqlOptions, restrictedToMinimumLevel: logLevel,
            columnOptions: ColumnOptions);
}

```

```

if (builder.Environment.IsDevelopment())
{
    SelfLog.Enable(msg =>
    {
        Debug.Print(msg);
        Debugger.Break();
    });
}
builder.Logging.AddSerilog(log.CreateLogger());
}
internal static void ValidateSettings(AppLoggerSettings settings)
{
    if (settings is null)
    {
        throw new CustomException(
            $"Configuration section '{nameof(AppLoggerSettings)}' is missing or could not be bound.");
    }
    List<string> validationErrors = [ ];
    ValidateObject(settings, nameof(AppLoggerSettings), validationErrors);
    if (settings.General is not null)
    {
        ValidateObject(settings.General,
            $"{nameof(AppLoggerSettings)}.{nameof(AppLoggerSettings.General)}", validationErrors);
    }
    if (settings.File is not null)
    {
        ValidateObject(settings.File,
            $"{nameof(AppLoggerSettings)}.{nameof(AppLoggerSettings.File)}", validationErrors);
    }
    if (settings.MsSqlServer is not null)
    {
        ValidateObject(settings.MsSqlServer,
            $"{nameof(AppLoggerSettings)}.{nameof(AppLoggerSettings.MsSqlServer)}", validationErrors);
    }
    if (validationErrors.Count > 0)
    {
        throw new CustomException(
            $"Configuration section '{nameof(AppLoggerSettings)}' is invalid: {string.Join("; ",
validationErrors)}");
    }
}
internal static void ValidateObject(object instance, string propertyPath,
    List<string> validationErrors)
{
    List<ValidationResult> validationResults = [ ];
    Validator.TryValidateObject(instance, new ValidationContext(instance), validationResults,
        validateAllProperties: true);

    foreach (ValidationResult validationResult in validationResults)
    {
        bool hasMemberName = false;
        foreach (string memberName in validationResult.MemberNames)
        {
            validationErrors.Add($"{propertyPath}.{memberName}: {validationResult.ErrorMessage}");
            hasMemberName = true;
        }
    }
}

```

```
        if (!hasMemberName)
        {
            validationErrors.Add($"{propertyPath}: {validationResult.ErrorMessage}");
        }
    }
}
```

Summary

This lab created the logging infrastructure and additional shared code for later labs.

Next steps

In the next part of this tutorial series, you will update the configuration settings for the ASP.NET Core application.